

Introducao ao Redis para Sistemas Distribuidos

Christian Aguiar Plentz¹, João Roberto Lemos Fidellis²

¹Ciência da Computação – Universidade do Vale do Rio dos Sinos (UNISINOS)
Caixa Postal 275 – 93022-750 – São Leopoldo – RS – Brazil

{plentzchristian, JRFIDELLIS}@edu.unisinos.br

Abstract. *This article presents the perks and benefits of using Redis for Distributed Systems compared to it's contemporaries*

Resumo. *Este artigo apresenta os atributos e benefícios em usar o Redis com a finalidade em Sistemas Distribuídos comparado aos seus contemporâneos*

1. Resumo sobre o Redis

O Redis é um banco de dados com foco em chave-valor que tem como sua principal vantagem sua velocidade ultra rápida para requisicao de dados, pois além de sua estrutura de chaves e valores também tem um foco em armazenamento do banco na memoria RAM, seu espaco em Sistemas Distribuidos é então em oferecer um serviço Rápido com Pouca latência entre os Serviços que se comunicam.

2. Casos de Uso

2.1. Cache em API Gateway

Um dos casos de uso mais comuns do Redis é como camada de cache em API Gateways. Neste cenário, o Redis atua com:

2.1.1. Redução de Latência

Cliente → API Gateway → Redis Cache → Microserviços
(valida sessão) (session store)

O gateway consulta o Redis para validar sessões e tokens de autenticação, evitando chamadas desnecessárias aos serviços backend. A latência típica de autenticação reduz de dezenas de milissegundos para microssegundos.

2.1.2. Proteção contra DDoS

redis-cli:

```
# Controle de requisições por IP
INCR ip:limit:192.168.1.100
EXPIRE ip:limit:192.168.1.100 60
```

```
# Se ultrapassar limite, bloquear acesso
if GET ip:limit:192.168.1.100 > 100 then
    return "Rate limit exceeded"
```

O Redis armazena contadores distribuídos para implementar rate limiting global, protegendo os servicos do backend de sobrecarga

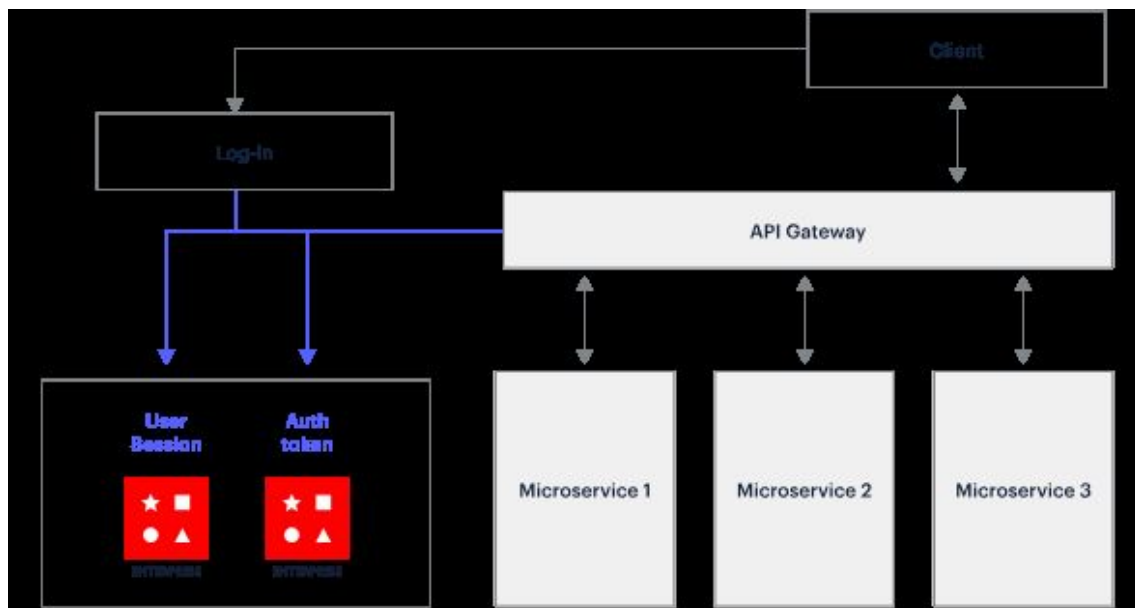


Figure 1. Modelo API Gateway

2.2. Cache de Banco de Dados

O padrão **CQRS** (Command Query Responsibility Segregation) com Redis como cache de leitura:

- **Commands:** Escritas diretas no banco de dados principal
- **Queries:** Leituras do cache Redis
- **CDC:** Sincronização assíncrona via Change Data Capture
- **Consistency eventual:** Dados atualizados periodicamente no cache

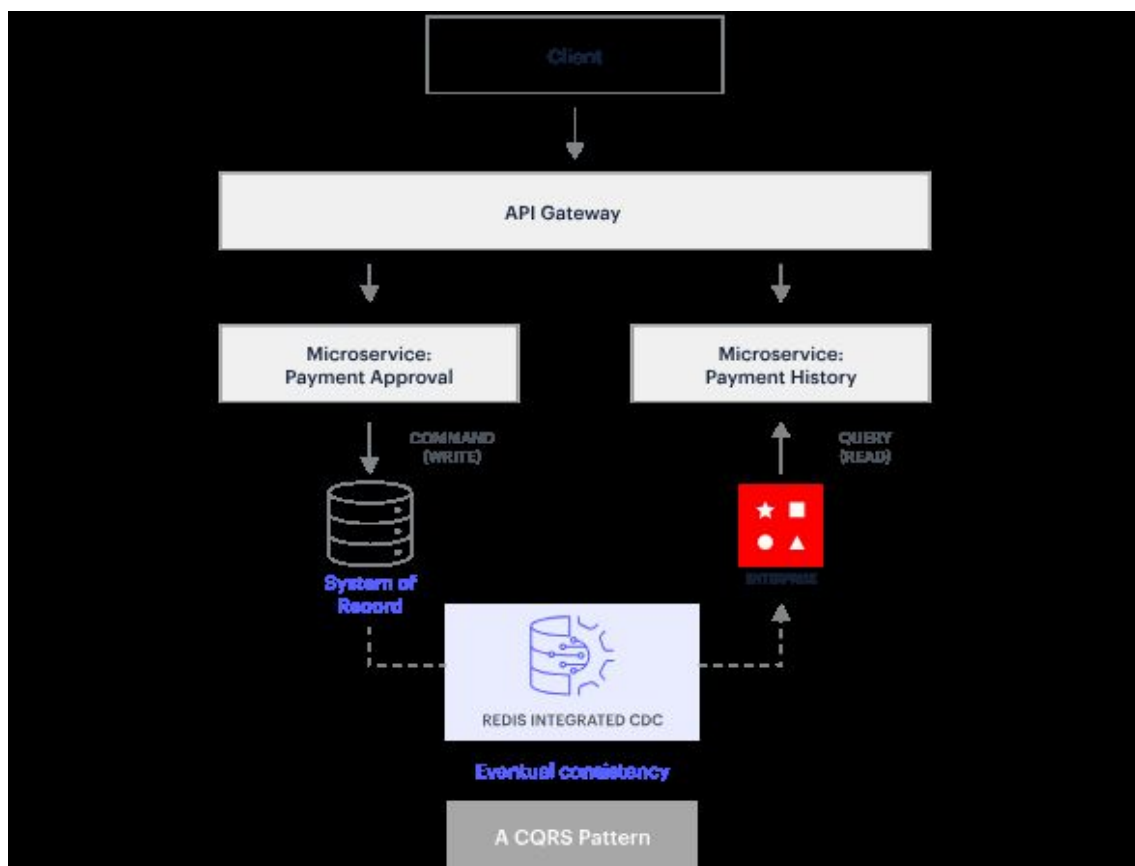


Figure 2. Padrão CQRS com Redis Cache

2.3. Sistema de Mensagens Assíncrona

O Redis oferece a Abordagem Pub/Sub como uma das suas frentes em Sistemas Distribuídos

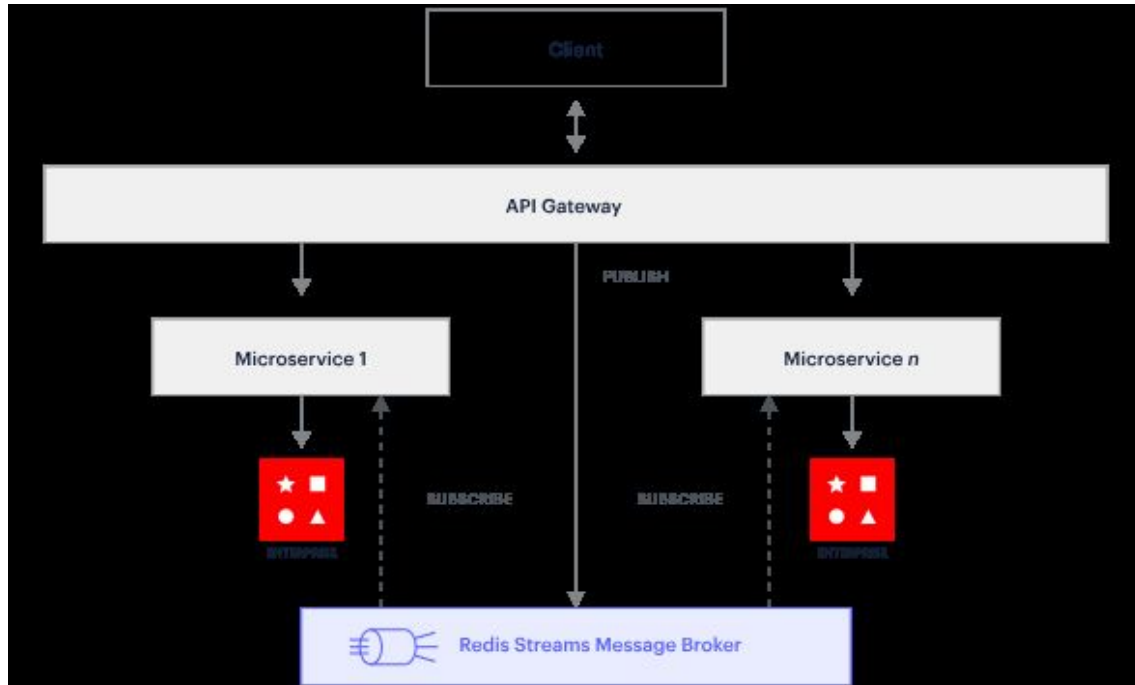


Figure 3. Comunicação Pub/Sub Redis

a API Gateway do Redis que se encarrega de enviar as mensagens de publish para o broker e as APIs de cada serviço se encarregam de se inscrever para esses tópicos que estão para ser utilizados.

3. Redis Sentinel

3.1. Arquitetura

O Redis Sentinel é um sistema distribuído projetado para monitoramento e failover automático de instâncias Redis. Sua arquitetura consiste em múltiplos processos Sentinel que cooperam para garantir alta disponibilidade.

3.2. Componentes Principais

- **Instância Mestre:** Nó primário que aceita escritas
- **Instâncias Escravas:** Réplicas sincronizadas do mestre
- **Processos Sentinel:** Múltiplas instâncias para consenso

3.3. Fluxo de Failover

1. **Deteção:** Sentinels detectam mestre indisponível
2. **Quórum:** Mínimo de Sentinels confirma falha
3. **Eleição:** Líder Sentinel é eleito para coordenar
4. **Promoção:** Melhor escravo é promovido a mestre
5. **Reconfiguração:** Clientes são notificados do novo mestre

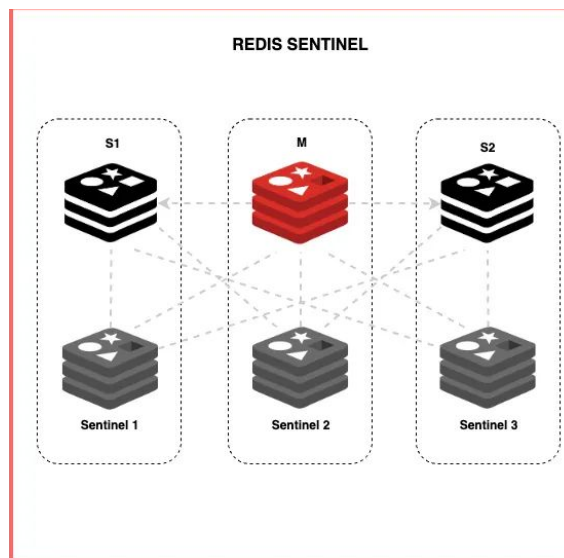


Figure 4. Infografico Redis Sentinel

4. Redis Cluster

4.1. Arquitetura e Particionamento

O Redis Cluster implementa sharding automático através de hash slots, distribuindo dados entre múltiplos nós mestres.

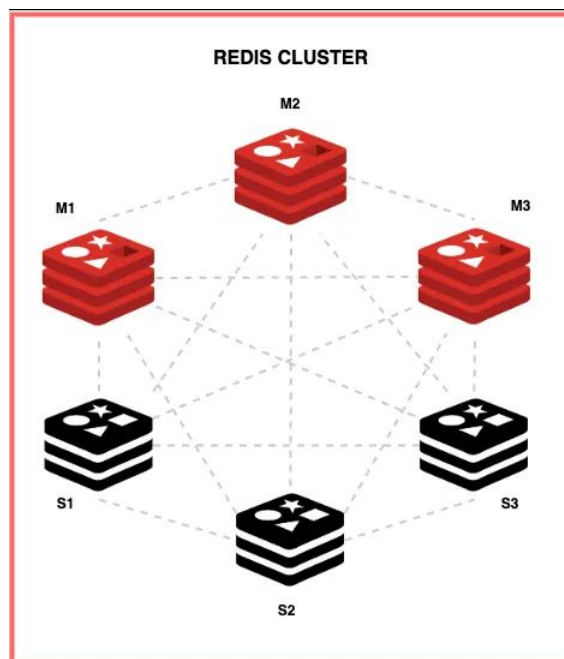


Figure 5. Infográfico Redis Cluster

4.2. Hash Slots e Sharding

O Redis Cluster implementa sharding automático através de um sistema de hash slots distribuídos entre múltiplos nós mestres. A arquitetura é projetada para fornecer escalabilidade horizontal e alta disponibilidade através de replicação.